

python 梯度下降实验报告

一、实验目的

1. 了解实现梯度下降的同时实现梯度下降的可视化过程
2. 加强对梯度下降的了解，锻炼个人代码能力。

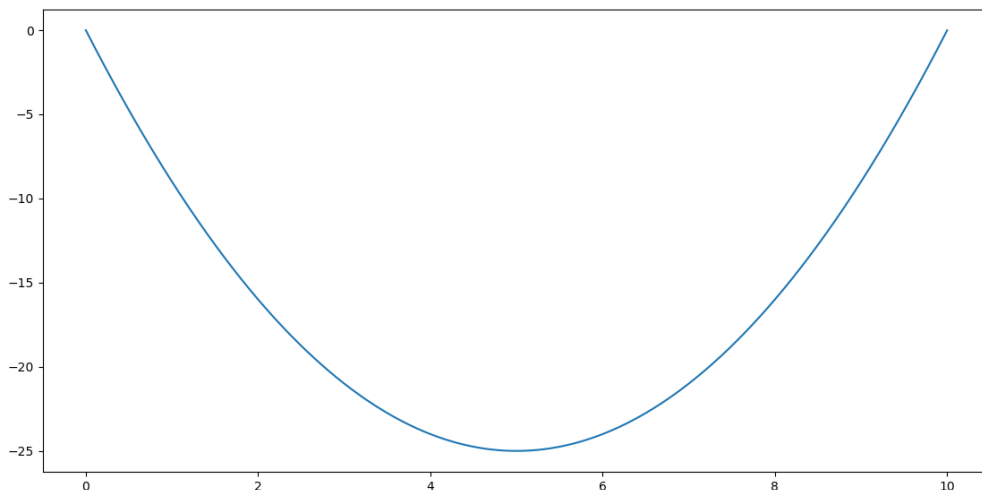
二、实验原理

梯度下降法在统计学习还是深度学习中都得到了广泛的应用，我们通过构建合理的模型，将模型预测的输出和正确结果进行误差计算，通过优化模型参数使得输出的结果和正确结果的误差最小，我们往往会把误差损失函数构建成一个凸函数，这样使得参数的求解比较单一化，就像一元二次函数只有一个波峰或者波谷，那么求解答案就是寻找这个波峰或者波谷，如果把误差损失函数表达成类似 $\sin x$ 的函数，存在多个波峰波谷，那么求解的答案就会有多个，显然求解的众多个答案并非都是最优解。

我们现在来解释梯度下降法。
以一个一元二次函数举例。

$$y = -10x + x^2 \quad y = -10x + x^2$$

画图如下：



假设这个表示我们模型的损失函数，现在我们的波谷，就是我们模型误差最小的参数点，此时认为 x 就是我们未知的参数，如何寻找 x 取值为多少才能达到这个波谷呢？

这就需要用到梯度下降法。

我们先对关于 x 的函数 y 求关于 x 的导函数，得到为

$$dy = -10 + 2x \quad dy = -10 + 2x$$

于是我们初始化 x 为一个随机值 a

$x=a$

然后不断迭代下面的过程

```
x1=x0-dy=x0-lr*(-10+2*x0)
x0=x1
```

上面的 lr 为学习率，也叫步长，意思为每次 x 更新变动的数据大小。因为这是一个不断迭代的过程，我们有两种方法让这个迭代结束，一种是设置误差方法，让这个误差小于某个阈值就结束，当然这种方法容易陷入死循环，因为你无法判断一个模型最终拟合的数据误差有多大，第二种方法就是设置迭代次数，这也是比较合理的。目前为止深度学习还是机器学习都比较常用的迭代结束判断条件。

现在解释下上面的迭代函数为什么可以找到这个波谷的近似位置，原因是，比如在波谷右边，波谷右边的函数曲线上的任意点斜率大于 0，此时 $x1=x0-lr*dy$ ，会让点 x 向左移，直到找到左边的波谷位置，如果在波谷左边，波谷左边的函数曲线斜率小于 0，那么 $x1=x0-lr*dy$ ，就会使得 x 数值增大，位置向右移，不断移动找到右边的波谷位置。

先利用我们的一元二次函数顶点公式求解出顶点的 $x=5$ ，现在利用梯度下降法去找一下这个结果是否和我们的标准结果一致。

三、程序

代码实现

```
import matplotlib.pyplot as plt
import numpy as np
import random
step=10000#设置迭代次数为 10000
x0=random.randint(-10,10)#随机初始化一个数值
lr=0.001#设置学习率为 0.001
for i in range(step):
    x1=x0-lr*(-10+2*x0)#梯度下降迭代
    x0=x1
print('梯度下降法求解结果:',x0)
```

四、程序运行结果图

每次运行结果不一致，因为 $x0$ 是随机初始化的一个数值，参数初始化对结果有影响。

```

41 ns [80] import matplotlib.pyplot as plt
import numpy as np
import random
step=10000#设置迭代次数为10000
x0=random.randint(-10,10)#随机初始化一个数值
lr=0.001#设置学习率为0.001
for i in range(step):
    x1=x0-lr*(-10+2*x0)#梯度下降迭代
    x0=x1
print('梯度下降法求解结果: ',x0)

```

梯度下降法求解结果: 5.000000002020285

```

75 ms 75 import matplotlib.pyplot as plt
import numpy as np
import random
step=10000#设置迭代次数为10000
x0=random.randint(-10,10)#随机初始化一个数值
lr=0.001#设置学习率为0.001
for i in range(step):
    x1=x0-lr*(-10+2*x0)#梯度下降迭代
    x0=x1
print('梯度下降法求解结果: ',x0)

```

✕ 梯度下降法求解结果: 5.0

```

33 ns [76] import matplotlib.pyplot as plt
import numpy as np
import random
step=10000#设置迭代次数为10000
x0=random.randint(-10,10)#随机初始化一个数值
lr=0.001#设置学习率为0.001
for i in range(step):
    x1=x0-lr*(-10+2*x0)#梯度下降迭代
    x0=x1
print('梯度下降法求解结果: ',x0)

```

梯度下降法求解结果: 4.999999995959427

五、实验结论

首先本实验选择的损失函数是凸函数，只有一个最低点。那么影响找到全局最小的损失的因素主要是学习率，训练次数以及参数的初始化。

经过修改学习率，我发现当学习率较大时会导致损失可能变大，当学习率过小时，会导致学习的过程十分缓慢。

训练次数也会影响训练的效果，当训练次数较少时，得不到好的训练效果，

当训练次数较大时，训练时间长。

最后就是参数的初始化，初始点没选好，可能会导致初始损失大，需要训练较长的训练时间才能得到较好的效果。